

# RELATÓRIO: RESERVA DE SALA

**Equipa:** Jamille Bezerra Candido, Francisco Vitor Gomes

**URL do Repositório:** [GitHub](#)

Neste trabalho, implementamos o sistema de reservas do Trabalho 1, substituindo a comunicação direta baseada em Sockets (TCP/UDP) por uma arquitetura baseada em Invocação Remota de Métodos (RMI). Para garantir total transparência de rede e cumprir a diretriz de não instanciar sockets nas aplicações cliente e servidor, adotamos a API nativa do Java RMI (*java.rmi.\**). Toda a lógica de comunicação e localização de objetos remotos foi abstraída.

O sistema foi estruturado de forma a separar completamente a lógica de negócio da infraestrutura de rede, utilizando a API nativa do Java RMI (*java.rmi.\**):

- **O Servidor:** No *ServidorEspacoFisico*, criámos um *registry* RMI local (porta 1099) utilizando *LocateRegistry.createRegistry(1099)*. Em seguida, registámos as instâncias das nossas classes de serviço (*ServicoReservaImpl* e *ServicoConsultaImpl*), assim como a nossa classe central *MiddlewareRMI*.
- **O Cliente e Middleware:** Para o *ClienteEspacoFisico*, o cliente invoca a operação local *middleware.doOperation(...)*. Internamente, o cliente obtém a referência remota do *MiddlewareRemoto* através do *Naming.lookup* e envia o pacote de bytes pela rede. O lado servidor do *Middleware* recebe o pacote, identifica o serviço de destino na sua lógica interna e invoca a operação correspondente.

O nosso modelo de dados foi expandido para cumprir integralmente os requisitos de estruturação em Orientação a Objetos:

- **Entidades (Mínimo de 4):** O sistema conta com 5 classes de entidade (*Sala*, *Laboratorio*, *Auditorio*, *Campus* e *Usuario*).
- **Composição por Agregação ("tem-um"):** Implementamos duas agregações. A classe *Campus* tem uma lista de *EspacoFisico*. Adicionalmente, a classe *ReservaAgendada* tem um *Usuario* e tem um *EspacoFisico*, representando a ligação entre o solicitante e o local.
- **Composição por Extensão ("é-um"):** As classes *Sala*, *Laboratorio* e *Auditorio* herdam (com a palavra *extends*) os atributos base da classe abstrata *EspacoFisico*.

O uso da API *java.rmi* permitiu aplicar as exigências de passagem de parâmetros de forma nativa e robusta:

- **Passagem por Referência para Objetos Remotos:** Cumprimos este requisito ao definir interfaces que herdam de *java.rmi.Remote* (*ServicoReservaRemote* e *ServicoConsultaRemote*). As implementações destas interfaces estendem *UnicastRemoteObject*. Quando o cliente faz o *Naming.lookup*, o Java RMI não copia as classes para o cliente, mas entrega-lhe apenas uma referência (um *Stub*), garantindo que a execução ocorre de facto remotamente no servidor.
- **Passagem por Valor e Representação Externa:** Para os objetos locais do servidor e os dados a enviar pela rede, adotamos o formato JSON construído nativamente. O

cliente instancia um objeto (por exemplo, a Sala), extrai os seus dados e o nosso sistema serializa isso numa String. Na classe MensagemRPC, dispensamos bibliotecas externas (como o Gson) e construímos a montagem e desmontagem do pacote utilizando concatenação, expressões regulares (Matcher e Pattern) e codificação em Base64. O RMI passa este pacote em formato de Array de Bytes (byte[]) por valor através da rede.

Para cumprir o requisito de fornecer, no mínimo, 4 métodos remotos, dividimos a lógica de negócio em dois serviços distintos que expõem as seguintes operações através da infraestrutura RMI:

1. **solicitarReserva**
2. **cancelarReserva**
3. **verificarDisponibilidade**
4. **consultarDetalhes**

O ClienteEspacoFisico foi programado para testar e validar o retorno bem-sucedido de cada um destes quatro métodos remotamente.

A adoção das bibliotecas nativas java.rmi.\* proporcionou uma solução robusta, onde a complexidade de rede (sockets, buffers, streams) foi completamente absorvida pelo *framework* nativo da linguagem. A integração com JSON para a passagem de parâmetros por valor complementou a arquitetura perfeitamente, atendendo a todos os requisitos exigidos.